

PRÁCTICA INTEGRAL DE POO EN PYTHON: 30 EJERCICIOS PROGRESIVOS

ENUNCIADOS DE LOS EJERCICIOS

NIVEL 1: FUNDAMENTOS (Ejercicios 1-10)

Objetivo: Dominar clases básicas, atributos y métodos

1. Clase Persona básica: Crear una clase `Persona` con atributos `nombre`, `edad` y `dni`. Implementar métodos `mostrar()` para mostrar datos y `es_mayor_de_edad()` que devuelva `True/False`.
2. Cuenta Bancaria: Crear clase `CuentaBancaria` con atributos `titular` (`Persona`), `saldo` (privado) y `número_cuenta`. Métodos: `depositar(monto)`, `retirar(monto)` (validar saldo) y `mostrar_saldo()`.
3. Rectángulo con propiedades: Clase `Rectangulo` con atributos `largo` y `ancho` (privados). Métodos: `calcular_area()`, `calcular_perimetro()`. Usar getters y setters para los atributos.
4. Círculo vs Cilindro: Clase `Circulo` con `radio` y método `area()`. Clase `Cilindro` que hereda de `Circulo` y agrega `altura` con método `volumen()`.
5. Estudiante con notas: Clase `Estudiante` con `nombre` y lista de notas. Métodos: `agregar_nota()`, `promedio()`, `mejor_nota()`, `peor_nota()`.
6. Libro con atributos estáticos:

Clase `Libro` con `título`, `autor`, `año`.

Atributo de clase `contador_libros` que lleve la cuenta de libros creados.

ISBN único auto-generado basado en `contador_libros` (ej: "LIB-0001").

Método `mostrar_info()` que muestre todos los datos del libro.

Ejemplo de uso:

```
libro1 = Libro("Cien años de soledad", "García Márquez", 1967)
```

```
libro2 = Libro("1984", "George Orwell", 1949)
```

```
print(libro1.isbn) # "LIB-0001"
```

```
print(libro2.isbn) # "LIB-0002"
```

```
print(Libro.contador_libros) # 2
```

7. Clase `Producto` con propiedades y métodos de clase

Crear clase `Producto` para una tienda:

ATRIBUTOS (con validación):

- codigo: auto-generado (ej: "PROD-001"), solo lectura
- nombre: string
- precio_base: > 0 (validar con @property)
- stock: ≥ 0 (validar con @property)

ATRIBUTO DE CLASE:

- IVA: 0.21 (21%) - constante para todos
- contador: cuenta productos creados

MÉTODOS:

- precio_final(): precio_base + IVA
- aplicar_descuento(%): reduce precio_base
- vender(cantidad): resta stock si hay suficiente
- __str__(): muestra código, nombre, precio y stock

MÉTODOS DE CLASE:

- crear_sin_stock(nombre, precio): crea con stock 0
- cambiar_iva(nuevo_iva): modifica IVA general

COMPARACIÓN:

- __lt__(otro): True si precio menor

EJEMPLO:

```
p1 = Producto("Laptop", 1000, 5)
p2 = Producto.crear_sin_stock("Mouse", 50)
p1.aplicar_descuento(10) # 10% descuento
print(p1) # PROD-001: Laptop - $900.00 (5 unidades)
print(p1.precio_final()) # $1089.00 (con IVA 21%)
print(p1 < p2) # False (900 < 50? No)
```

8. Sistema de Inscripciones a Eventos

Crear clase Inscripcion que maneje fechas usando datetime.

REQUISITOS:

1. Atributos:

- nombre_participante (str)
- email (str)
- fecha_inscripcion (datetime.date) → usar datetime para fecha actual
- fecha_evento (datetime.date)

2. Validaciones con datetime:

- La fecha de evento debe ser FUTURA (mayor a fecha actual)
- La inscripción debe hacerse AL MENOS 3 días antes del evento

3. Métodos usando datetime:

- dias_para_evento(): días faltantes (fecha_evento - fecha_actual)
- es_inscripcion_valida(): True si cumple validaciones
- cambiar_fecha_evento(nueva_fecha): validar que sea futura

4. Métodos de formato:

- mostrar_info(): muestra datos con fechas formateadas

5. Método de clase:

- crear_desde_formulario(nombre, email, fecha_evento_str):
recibe fecha como string "dd/mm/aaaa" y la convierte con strptime

EJEMPLO:

```
insc = Inscripcion("Ana", "ana@email.com", "15/12/2024")
print(insc.dias_para_evento()) # días faltantes
print(insc.es_inscripcion_valida()) # True si válida
print(insc.mostrar_info())
```

9. Calculadora Básica con Historial

Objetivo: Practicar encapsulamiento y listas como atributos privados.

Crear una clase Calculadora que tenga:

- Un atributo privado `__historial` (lista) que almacene las últimas 5 operaciones realizadas
- Métodos para operaciones básicas:
 - `sumar(a, b)`
 - `restar(a, b)`
 - `multiplicar(a, b)`
 - `dividir(a, b)` (validar división por cero)
- Método `mostrar_historial()` que muestre las operaciones guardadas
- Cada operación debe guardarse como string en el historial (ej: "5 + 3 = 8")
- El historial solo guarda máximo 5 operaciones (eliminar la más antigua si hay más)

Ejemplo:

```
python
calc = Calculadora()
calc.sumar(5, 3)      # Guarda "5 + 3 = 8"
calc.multiplicar(4, 2) # Guarda "4 × 2 = 8"
calc.dividir(10, 2)   # Guarda "10 ÷ 2 = 5.0"
calc.mostrar_historial() # Muestra las 3 operaciones
```

10. Gestor de Contactos Simple

Objetivo: Practicar diccionarios como estructura de datos y validaciones básicas.

Crear una clase AgendaContactos que maneje una lista de contactos donde cada contacto es un diccionario con:

- nombre (string, obligatorio)
- telefono (string de 10 dígitos, validar)

- email (string, debe contener "@", validar formato básico)

Métodos:

- agregar_contacto(nombre, telefono, email): Valida datos y agrega contacto
- buscar_contacto(nombre): Devuelve contacto por nombre o None
- eliminar_contacto(nombre): Elimina contacto si existe
- mostrar_contactos(): Muestra todos los contactos ordenados alfabéticamente
- exportar_contactos(): Devuelve lista de contactos (para posible guardado en archivo)

Ejemplo:

python

```
agenda = AgendaContactos()
```

```
agenda.agregar_contacto("Ana", "1122334455", "ana@email.com")
```

```
agenda.agregar_contacto("Carlos", "9988776655", "carlos@empresa.com")
```

```
print(agenda.buscar_contacto("Ana")) # Muestra datos de Ana
```

```
agenda.mostrar_contactos() # Muestra todos ordenados
```

NIVEL 2: HERENCIA Y POLIMORFISMO (Ejercicios 11-20)

Objetivo: Dominar herencia, sobrescritura y polimorfismo

- Figuras geométricas: Crear una jerarquía de clases que implemente diferentes figuras geométricas aplicando herencia y polimorfismo. Deberás desarrollar una clase base Figura con métodos para calcular área y perímetro, y luego crear clases derivadas Cuadrado, Rectángulo, Círculo y Triángulo que sobrescriban estos métodos con sus fórmulas específicas. Finalmente, demostrarás el polimorfismo creando una lista de figuras de diferentes tipos y calculando áreas y perímetros totales de manera unificada.
- Crea un sistema de nómina usando herencia y polimorfismo. Implementa una clase base Empleado con atributos nombre, id_empleado y un método calcular_salario(). Luego crea las clases EmpleadoTiempoCompleto (con salario mensual y bono), EmpleadoMedioTiempo (con horas semanales y tarifa por hora) y EmpleadoPorProyecto (con proyectos completados y pago por proyecto), que hereden de Empleado y sobrescriban el método calcular_salario() según su fórmula específica. Además, cada clase debe tener un método trabajar() que describa su jornada laboral. Finalmente, demuestra el polimorfismo creando una lista con diferentes tipos de empleados, calculando la nómina total y mostrando información de cada empleado.
- Crea un sistema de vehículos usando herencia donde diferentes tipos de vehículos calculen su consumo de combustible de manera distinta. Implementa una clase base Vehiculo con atributos marca, modelo y año, y un método calcular_consumo(kilometros) que devuelva el consumo genérico.

Luego crea las clases Auto, Moto y Camion que hereden de Vehiculo y sobrescriban el método `calcular_consumo()` con sus fórmulas específicas: Auto 12 km/l, Moto 25 km/l, Camión 5 km/l. Finalmente, demuestra el polimorfismo creando una lista con diferentes vehículos y calculando el consumo total para un viaje de 100 km.

14. Crea un sistema de animales usando herencia y polimorfismo. Implementa una clase base Animal con atributos nombre y edad, y un método `hacer_sonido()` que devuelva un string genérico. Luego crea las clases Perro, Gato y Vaca que hereden de Animal y sobrescriban el método `hacer_sonido()` con sus sonidos específicos: Perro "¡Guau guau!", Gato "¡Miau miau!", Vaca "¡Muuu!". Finalmente, demuestra el polimorfismo creando una lista con diferentes animales y haciendo que cada uno emita su sonido característico.
15. Crea un sistema de pagos usando herencia y polimorfismo. Implementa una clase base Pago con atributos de solo lectura: monto y procesado, y un método `procesar_pago()` que debe ser sobrescrito. Luego crea las clases TarjetaCredito, PayPal y TransferenciaBancaria que hereden de Pago y sobrescriban `procesar_pago()` con su lógica específica. Incluye también validaciones apropiadas para cada tipo de pago. Finalmente, demuestra el polimorfismo creando una lista con diferentes pagos y procesándolos de manera unificada.

Requisitos:

- Los atributos monto y procesado deben ser de solo lectura
- Cada tipo de pago debe tener validaciones específicas
- El método `procesar_pago()` debe actualizar el estado

16. Crea un sistema de notificaciones usando herencia y polimorfismo. Implementa una clase base Notificacion con un método `enviar(mensaje)` y atributos id (único y auto-generado) y timestamp. Luego crea las clases Email, SMS y PushNotification que hereden de Notificacion y sobrescriban el método `enviar()` con su lógica específica. Asegúrate que cada notificación tenga un ID único. Finalmente, demuestra el polimorfismo creando una lista con diferentes tipos de notificaciones y enviándolas de manera unificada.

Requisitos adicionales:

- Cada notificación debe tener un id único auto-generado
- Incluir timestamp de creación
- Validaciones básicas para cada tipo

17. Crea un sistema de instrumentos musicales usando herencia y polimorfismo. Implementa una clase base Instrumento con atributos nombre y afinado, y un método `tocar()` que devuelva un sonido genérico. Luego crea las clases Guitarra, Piano y Bateria que hereden de Instrumento y sobrescriban el método `tocar()` con su sonido específico. Incluye también un método `afinar()` que cambie el estado a afinado. Finalmente, demuestra el polimorfismo creando una lista "orquesta" con diferentes instrumentos y haciendo que cada uno toque de manera unificada..

18. Crea un sistema de conversión de documentos usando herencia y polimorfismo. Implementa una clase base Documento con atributos titulo, contenido y tamaño, y un método exportar(formato) que convierta el documento a diferentes formatos. Luego crea clases específicas para cada tipo de documento (PDFDocumento, WordDocumento, HTMLDocumento) que hereden de Documento y tengan métodos adicionales específicos de su formato. Incluye un método de clase convertir() que pueda convertir entre formatos. Finalmente, demuestra el polimorfismo creando un sistema donde un mismo contenido pueda exportarse a múltiples formatos.

Característica nueva: Métodos de clase para conversión entre formatos.

19. Sistema de Figuras 3D con Clase Abstracta: Crea un sistema de figuras geométricas 3D usando clases abstractas, herencia y polimorfismo. Implementa una clase abstracta Figura3D con métodos abstractos volumen() y area_superficial(), y luego crea las clases concretas Cubo, Esfera y Cilindro que hereden de ella e implementen estos métodos con sus fórmulas correspondientes. Finalmente, demuestra el polimorfismo creando una lista con diferentes figuras 3D y calculando volúmenes y áreas superficiales de manera unificada
20. Crea un sistema educativo usando herencia y polimorfismo. Implementa una clase base Persona con atributos nombre, edad y método presentarse(). Luego crea las clases Estudiante y Profesor que hereden de Persona y sobrescriban el método presentarse(). Cada una debe tener métodos específicos: estudiar(materia) para estudiantes y enseñar(materia) para profesores. Finalmente, demuestra el polimorfismo creando una lista con diferentes personas y haciendo que cada una se presente y realice su actividad específica de manera unificada.

NIVEL 3: COMPOSICIÓN Y AGREGACIÓN (Ejercicios 21-30)

Objetivo: Dominar relaciones entre objetos

EJERCICIO 21: Biblioteca con Composición

Crea un sistema de biblioteca usando composición. Implementa una clase Biblioteca que contenga una lista de objetos Libro (relación de composición: los libros existen mientras exista la biblioteca). La clase Libro debe tener título, autor y año. La Biblioteca debe tener métodos para agregar libros, buscar libros por autor, y listar libros prestados. Los libros prestados deben marcarse con un atributo prestado.

EJERCICIO 22: Universidad con Agregación

Crea un sistema universitario usando agregación. Implementa las clases Universidad, Facultad, Carrera y Estudiante donde:

- Una Universidad agrega varias Facultad (pero las facultades pueden existir sin la universidad)
 - Una Facultad agrega varias Carrera
 - Una Carrera agrega varios Estudiante
- Incluye métodos para contar estudiantes por facultad y mostrar la estructura completa.

EJERCICIO 23: Carrito de Compras

Crea un sistema de carrito de compras usando agregación. Implementa las clases Producto (con nombre, precio y código) y Carrito. El Carrito debe contener una lista de Producto con sus cantidades. Incluye métodos para agregar productos, calcular el total, aplicar descuentos y mostrar el contenido del carrito.

EJERCICIO 24: Equipo Deportivo

Crea un sistema de equipo deportivo usando agregación. Implementa las clases Jugador (con nombre, edad y posición) y Equipo. El Equipo debe contener una lista de Jugador. Incluye métodos para contratar jugadores, calcular el promedio de edad, y encontrar el jugador más valioso (simulado por un atributo de valoración).

EJERCICIO 25: Casa con Habitaciones

Crea un sistema de vivienda usando composición. Implementa una clase Casa que esté compuesta por varias Habitación. Cada Habitación debe tener tipo (dormitorio, cocina, baño) y tamaño (m²). La Casa debe tener un método para calcular el área total sumando las áreas de todas las habitaciones.

EJERCICIO 26: Orden de Pedido

Crea un sistema de pedidos usando composición. Implementa las clases Orden, ItemOrden y Producto. Cada Orden está compuesta por varios ItemOrden, y cada ItemOrden referencia un Producto con una cantidad. Incluye métodos para calcular el total del pedido, aplicar descuentos por cantidad, y mostrar el detalle del pedido.

EJERCICIO 27: Red Social

Crea un sistema básico de red social usando agregación y composición. Implementa las clases `Usuario`, `Amigo` (agregación) y `Publicacion con Comentario` (composición). Un `Usuario` puede tener múltiples `Amigo` (otros usuarios), y puede crear `Publicacion` que contengan múltiples `Comentario`. Incluye métodos para agregar amigos, publicar y comentar.

EJERCICIO 28: Hospital con Composición

Crea un sistema hospitalario usando composición. Implementa las clases `Hospital`, `Departamento`, `Doctor` y `Paciente`. Un `Hospital` está compuesto por varios `Departamento`, cada `Departamento` contiene `Doctor` y `Paciente`. Incluye métodos para asignar pacientes a doctores, contar pacientes por departamento, y mostrar la estructura del hospital.

EJERCICIO 29: Vuelo y Pasajeros

Crea un sistema de aerolínea usando agregación. Implementa las clases `Vuelo`, `Pasajero` y `Equipaje`. Un `Vuelo` agrega varios `Pasajero`, y cada `Pasajero` tiene un `Equipaje`. Valida que el vuelo no exceda su capacidad máxima de pasajeros. Incluye métodos para agregar pasajeros, calcular el peso total del equipaje, y verificar disponibilidad.

EJERCICIO 30: Proyecto de Software

Crea un sistema de gestión de proyectos usando composición. Implementa las clases `Proyecto`, `Tarea` y `Desarrollador`. Un `Proyecto` está compuesto por varias `Tarea`, y cada `Tarea` tiene un `Desarrollador` asignado. Incluye métodos para calcular la duración total del proyecto, listar los desarrolladores involucrados, y mostrar el progreso del proyecto.

EJERCICIOS INTEGRADORES FINALES (Bonus)

EJERCICIO 31: Sistema de Reservas de Hotel (Completo)

Crea un sistema integral de reservas de hotel que combine herencia, composición y polimorfismo. Implementa una jerarquía de herencia para los tipos de habitaciones (`Habitacion` → `Simple`, `Doble`, `Suite`), cada una con su propio método `calcular_precio()` que aplique polimorfismo. Luego crea la clase `Hotel` usando composición para contener las habitaciones. El sistema debe permitir hacer

reservas, calcular ingresos totales, y verificar disponibilidad de habitaciones por tipo y fecha.

EJERCICIO 32: Tienda Online Completa

Desarrolla una tienda online completa integrando todas las relaciones de POO. Crea las clases `Cliente`, `Producto`, `Categoria`, `Carrito` y `Orden`. Implementa herencia para tipos de productos (ej: `ProductoFisico`, `ProductoDigital`), agregación entre `Carrito` y `Producto`, y composición entre `Orden` y sus items. El sistema debe manejar el proceso completo: selección de productos, aplicación de descuentos, generación de órdenes y cálculo de totales.

EJERCICIO 33: Sistema Bancario Avanzado

Crea un sistema bancario complejo que combine herencia múltiple y agregación. Implementa una jerarquía de cuentas (`Cuenta` → `Ahorros`, `Corriente`, `PlazoFijo`) cada una con reglas específicas. Luego crea la clase `Banco` que use agregación para manejar `Cientes` y sus `Cuentas`. El sistema debe permitir transferencias entre cuentas (validando tipos), cálculo de intereses según el tipo de cuenta, y generar reportes financieros.

EJERCICIO 34: Sistema Educativo con Herencia Múltiple

Diseña un sistema educativo avanzado usando herencia múltiple. Crea las clases base `Persona`, `Estudiante` y `Trabajador`, y luego una clase `Asistente` que herede tanto de `Estudiante` como de `Trabajador`. Implementa interfaces con métodos abstractos para definir comportamientos obligatorios. El sistema debe manejar roles múltiples, calcular cargas horarias combinadas, y gestionar permisos según el tipo de usuario.

EJERCICIO 35: Juego de Cartas Estratégico

Desarrolla un juego de cartas estratégico que integre composición, herencia y polimorfismo. Crea las clases `Mazo` (compuesto por `Carta`), `Jugador` (hereda de `Persona`), y `Juego` que coordina todo. Implementa polimorfismo en las reglas del juego usando clases para diferentes tipos de cartas (`CartaAtaque`, `CartaDefensa`, `CartaEspecial`). El sistema debe manejar turnos, aplicar efectos polimórficos de las cartas, y determinar al ganador según diferentes estrategias.

ESTRUCTURA DE CADA EJERCICIO EN LA PRÁCTICA:

Para cada ejercicio se entregará:

1. Enunciado completo con especificaciones
2. Código base para completar
3. Solución comentada paso a paso
4. Explicación conceptual de lo que se enseña
5. Pruebas/Test cases para verificar
6. Variantes/challenges adicionales